

Lecture 2: Performance Optimization and Advanced Julia

Computational Bootcamp

John P. Ryan

Summer 2026

University of Wisconsin–Madison

Today's goals

1. Understand how your computer works and interacts with programming languages
2. Learn some advanced Julia concepts, especially to improve how fast your code runs
3. Learn how to *measure* performance so you're not guessing

How computers run your code

Machine language

- Computers are very good at performing calculations and executing algorithms
- Unfortunately, a computer's hardware only understands things written in machine language
- A machine language consists of instructions written using only binary 0's and 1's
- This machine language cannot easily be understood by humans

- Programming languages act as the translator between you and your computer
- You write a set of instructions in a programming language, such as Julia
- When you execute your code, these instructions get translated to machine language
- How fast your code runs will be impacted by:
 1. How you write your code
 2. How your instructions get translated to machine language
 3. What computer you are using

Compiled vs. interpreted languages

- Programming languages generally fall into two groups:
 1. Compiled languages (C, Fortran, Rust)
 - All of the code gets translated before execution
 - Generates very efficient machine code
 - Cannot code interactively or make changes on the fly
 2. Interpreted languages (Stata, R, Python, Matlab)
 - Translates each line of code just before its execution
 - This allows for dynamic and interactive coding
 - Will not generate efficient machine code

Just-in-time compilation

- Julia tries to have the best of both worlds with **just-in-time (JIT) compilation**
- When executing a function for the first time, Julia compiles it to machine language
- On subsequent calls, the compiled machine code is called directly instead of re-compiling
- This gives the interactivity of an interpreted language with (potentially) compiled-language speed
- Efficient machine code is only generated *if you follow best practices*:
 - <https://docs.julialang.org/en/v1/manual/performance-tips/>
- Practical consequence: the *first* call to a function includes compilation time; don't confuse this with slow code

The big performance rules

Overall performance tips

1. Don't use global variables — put code in **functions**
2. Declare types & write **type-stable** code: don't let a variable change type
3. Use **structs** to organize model primitives and results
4. **Pre-allocate** arrays that you fill repeatedly
5. Use **views** instead of slices to avoid copies
6. **Fuse dot operations** to avoid temporary arrays
7. Iterate in **column-major order**
8. **Benchmark and profile** instead of guessing

We'll take these one at a time.

1. Don't use globals; use functions

- A **global variable** is defined in the global scope: wherever you are in the program, Julia has to assume anything could have changed it
- Globals can change type over the course of execution, so Julia cannot generate optimized machine code in advance
- In contrast, Julia compiles a *specialized, efficient* version of each function based on the types of its arguments
- The same operation inside a function can be **orders of magnitude faster** than in global scope
- Bottom line: any code you want to be fast goes in a function; pass everything it needs as arguments

2. Type stability

- Every variable has a type (`Int64`, `Float64`, `String`, `Matrix{Float64}`, ...)
- If Julia can *infer* a single concrete type for each variable, it generates fast machine code; if the type can change, it must generate slow, generic code

```
x = 4           # Int64
x = x / 2       # now a Float64! Type of x changed -> slow
x = 4.0         # better: start as Float64 if division is coming
```

- Common culprits: initializing with 0 instead of 0.0, containers of mixed types, functions that return different types on different branches
- Write functions with type declarations:

```
function f(x::Float64, y::Float64)
    return x + y
end
```

3. Structs for model primitives

- Structs let you bundle parameters together with concrete types — fast *and* convenient
- Modern Julia has `@kwdef` built in (no need for the old `Parameters.jl` package):

```
@kwdef struct Primitives
    beta::Float64 = 0.99 # discount factor
    alpha::Float64 = 0.36 # capital share
    delta::Float64 = 0.025 # depreciation
    nk::Int64 = 1000 # grid points
    kgrid::Vector{Float64} = collect(range(0.01, 45.0, length=1000))
end

p = Primitives() # defaults
p2 = Primitives(beta = 0.95) # override one field
p.beta == 0.99 # true
(; beta, alpha) = p # destructure fields you need
p.beta = 0.95 # error: immutable struct
beta = 0.95 # This works - unpacked into broader scope
```

4. Pre-allocation

- If you repeatedly create temporary arrays inside a loop, you pay for memory allocation (and garbage collection) every iteration
- Instead: allocate the array *once*, then fill it in place

```
# Bad: allocates a new vector every iteration
```

```
for i in 1:1000
    tmp = zeros(n)
    ...
end
```

```
# Good: allocate once, overwrite in place
```

```
tmp = zeros(n)
for i in 1:1000
    tmp .= 0.0    # note the dot: in-place assignment
    ...
end
```

- Useful when you don't need results across iterations (e.g. per-iteration likelihood contributions you only sum)

5. Views instead of slices

- A slice like `V[:,1:5]` *copies* that block of the array into new memory
- A **view**, `view(V[:,1:5])` or `@views V[:,1:5]`, just refers to the existing memory — no copy
- If you only need to *read* a block inside a hot loop, copying is wasted time

```
sum(V[:, 1:5])           # allocates a copy, then sums
sum(@view V[:, 1:5])     # no allocation
@views mean(V[i, :])     # @views applies to a whole expression
```

6. Fuse dot operations

- Dots broadcast a function elementwise: `.*`, `.+`, `exp.(x)`, ...
- Julia **fuses** chains of dotted operations into a single loop with no temporaries
- If you mix dotted and un-dotted operators, you break the fusion:

```
z = x .+ y - w ./ 4    # computes x .+ y, w ./ 4 separately (temporaries!)
z = x .+ y .- w ./ 4   # one fused loop, no temporaries
z .= x .+ y .- w ./ 4  # even better: writes into existing z in place
@. z = x + y - w / 4   # @. dots everything for you
```

7. Order of iteration matters

- Julia stores arrays in **column-major order**: elements of a column are adjacent in memory
- So the *inner* loop should run down columns (first index fastest):

```
# Good: inner loop moves down a column
for j in 1:ncols, i in 1:nrows
    A[i, j] += 1.0
end

# Better yet: let Julia pick the fast order
for idx in eachindex(A)
    A[idx] += 1.0
end
```

- The speedup from fixing loop order on large matrices can be massive
- (This is the opposite convention from C and NumPy, which are row-major)

8. Measure, don't guess: BenchmarkTools.jl

- @time includes compilation the first time; use it only for long-running calls
- **BenchmarkTools.jl** runs your code many times and reports reliable statistics:

```
using BenchmarkTools
@btime solve_model($p)      # $ interpolates variables to avoid
                             # benchmarking global-variable access
```

- Watch the **allocations** count: unexpected allocations in a hot loop are usually the problem
- Workflow: get it *correct* first, then optimize your algorithm, then benchmark & optimize the bottleneck

Application: Optimal Investment

Optimal investment problem

- We'll apply these principles to a simple optimal investment (growth) problem
- Given capital k , we have resources

$$y = k^\alpha + (1 - \delta)k$$

- We live forever and discount the future at rate β . Utility from consumption is $\log(c)$
- We need to decide how much to consume (c) and how much capital to save for the next period (k'):

$$V(k) = \max_{c, k'} \{ \log(c) + \beta V(k') \}$$

$$c + k' = k^\alpha + (1 - \delta)k$$

- This is a contraction mapping, so we can iterate on V until convergence
- We proceed as follows:

1. Guess V_1 . Solve

$$V_2(k) = \max_{c,k'} \log(c) + \beta V_1(k')$$

2. Then, repeat this again:

$$V_3(k) = \max_{c,k'} \log(c) + \beta V_2(k')$$

3. We know that $V_n \rightarrow V$ as $n \rightarrow \infty$

- More on this later - to the code!